

SimDiff: Point Cloud Acceleration by Utilizing Spatial Similarity and Differential Execution

Yangfan Li¹, Mengquan Li¹, Cen Chen¹, *Senior Member, IEEE*, Xiaofeng Zou¹, Hongen Shao,
Fengxiao Tang¹, *Member, IEEE*, and Kenli Li¹, *Senior Member, IEEE*

Abstract—Point cloud neural networks are gaining increasing attention in emerging 3-D computer vision applications, such as autonomous driving, robotics, and virtual reality. Many customized accelerators for 3-D point clouds have been developed to pursue superior time and energy efficiencies. In this work, we reveal that spatially adjacent points in a 3-D point cloud show similar feature values and relationships, implying substantial redundant computations and memory accesses, while which have been previously ignored. To reduce such redundancies, we propose *SimDiff*, an algorithm-accelerator co-design framework that boosts 3-D point cloud processing by cleverly leveraging spatial similarity toward excellent speedup and energy efficiency. On the algorithm side, we design a novel similarity-aware differential point cloud neural network (dubbed *SD-PCNet*). Differing from the standard flow of mainstream point cloud networks, it abstracts a brand-new execution flow for point cloud processing by utilizing spatial similarity among points and dynamic differential execution. On the accelerator side, we propose *SD-PCAcc*, a supporting accelerator to convert algorithm-level redundancy reductions into performance enhancements. On the deployment side, we propose efficient strategies for network-to-accelerator mapping and scheduling, high-bandwidth memory (HBM) channel allocation, and core component reconfiguration, facilitating the proposed methodologies into practical implementation. Extensive evaluation results show that, with preserved accuracy, our *SimDiff* gains an average of 3.2× speedup and 3.1× energy efficiency compared to the state-of-the-art competitors.

Index Terms—Hardware accelerator, point cloud, software-hardware co-design, spatial similarity.

I. INTRODUCTION

WITH 3-D acquisition technologies advance, 3-D sensors like LiDARs and RGB-D cameras now become popular. 3-D vision data, especially point cloud (which is one of the most significant data formats for 3-D representation) generated by these sensors offers detailed geometric, shape, and scale information of objects. Compared to 2-D images, point clouds enhance a better understanding of real-world scenes, being widely applied in numerous fields, such as autonomous driving [1], robotics [2], and VR/AR [3].

To adapt well to the unstructured and unordered nature of 3-D point clouds, point-cloud neural networks emerged and gradually flourished. They are a type of deep neural network models that enable efficient point cloud processing. Compared to conventional methods that transformed point clouds into 2-D images [4] or 3-D voxels [5], recent progress in point-cloud network models directly process 3-D point clouds in a point-based pipeline manner [6], [7], [8], [9], [10] for higher accuracy. Among them, two popular models DGCNN [6] and PointNet++ [7] have been widely utilized, functioning as backbones.

However, these prominent models are computationally extensive and have expensive random memory accesses. Typically, for each point in a point cloud network, its standard processing flow can be abstracted as three stages: 1) Neighbor Search; 2) Edge Update; and 3) Neighbor Aggregation. The Neighbor Search stage is to find the neighbors of the target points in the cloud, based on the spatial feature distance among points. The Edge Update stage is to calculate the spatial relation between the target point and its neighbor points. In the Neighbor Aggregation stage, all neighbor information of the target point is aggregated into a single feature vector. Based on such a pipeline, there are a large amount of random memory accesses when collecting neighbor points' features in the Neighbor Search and Aggregation stages. In addition, the edge update operation will incur a lot of matrix multiplication calculations considering all the edges connecting the target points and their neighbors.

To support the efficient deployment of point cloud networks, numerous customized accelerators based on have been put forward [11], [12], [13]. They have designed various strategies to optimize the point-based pipeline from different angles.

Manuscript received 5 February 2024; revised 28 May 2024; accepted 23 July 2024. Date of publication 19 August 2024; date of current version 22 January 2025. This work was supported in part by the Young Scientists Fund of the National Natural Science Foundation of China under Grant 62302529; in part by the Hunan Provincial Natural Science Foundation of China under Grant 2023JJ40770; in part by the Changsha Municipal Natural Science Foundation under Grant kq2208290; in part by the National Natural Science Foundation of China under Grant 62302529 and Grant 62202159; in part by the Fundamental Research Funds for the Central Universities under Grant 2023ZYGXZR023; by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515010220; and by the Basic Research of Shenzhen Science and Technology Plan under Grant JCYJ20210324123802006. This article was recommended by Associate Editor G. Ansaloni. (*Corresponding author: Cen Chen.*)

Yangfan Li and Fengxiao Tang are with the School of Computer Science and Engineering, Central South University, Changsha 410075, China (e-mail: liyangfan37@csu.edu.cn; tangfengxiao@csu.edu.cn).

Mengquan Li is with the College of Integrated Circuits, Hunan University, Changsha 410082, China (e-mail: mengquanli@hnu.edu.cn).

Cen Chen is with the School of Future Technology, South China University of Technology, Guangzhou 510640, China, and also with the Shenzhen Institute, Hunan University, Shenzhen 518055, China (e-mail: chencen@scut.edu.cn).

Xiaofeng Zou and Hongen Shao are with the School of Future Technology, South China University of Technology, Guangzhou 510640, China (e-mail: chencen@scut.edu.cn; zouxiaofeng@scut.edu.cn; ftshaohongen@scut.edu.cn).

Kenli Li is with the College of Information Science and Engineering, Hunan University, Changsha 410082, China (e-mail: lk1@hnu.edu.cn).

Digital Object Identifier 10.1109/TCAD.2024.3445808

Crescent [13] employs approximation techniques to enhance the processing efficiency of irregular memory accesses inherent in point cloud networks. PointAcc [12] addresses the intrinsic sparsity of point clouds by implementing a caching mechanism that supports on-demand streaming of sparse calculations.

In this work, we attempt to explore highly efficient acceleration for 3-D point cloud from a new perspective as inspired by our recent findings from the processing flow of point cloud neural networks: we reveal that there are significant similarities and correlations among spatially adjacent points, and thus noticeable execution redundancy exists. We propose to further boost the speedup and energy efficiency of point cloud accelerators by reducing such redundancy.

We start with understanding the spatial similarity within point cloud networks. There are mainly two similarities among spatially adjacent points. First, the neighbors of a target point are usually in close spatial proximity and tend to have similar features. Second, spatially adjacent points commonly have a large amount of shared neighbors. These multiple similarities imply that when processing the points with spatial similarity, there must be similar execution procedures (and operations), resulting in execution redundancy. Once such redundancy is well eliminated, the processing efficiency of 3-D point clouds can be further improved significantly.

However, exploiting such spatial similarities to reduce redundancy is challenging. One key obstacle is that, edge update operations in the backbone models would break the feature similarity between adjacent points. When updating the feature of an edge (which connects a target point and one of its neighbors), there is a subtraction operation with nonlinear activation between their features. This operation could change the feature values of the two adjacent points and disrupt spatial feature similarity. Second, each edge of two points is updated independently, rendering distinct intermediate features that are hard to reuse for target points with shared neighbors. In addition, it is computationally and memory access costly to obtain the shared neighbors.

To this end, we develop *SimDiff*, an algorithm–hardware co-design framework that boosts 3-D point cloud processing by cleverly leveraging spatial similarity toward superior speedup and energy efficiency. On the algorithm side, we design a novel similarity-aware differential point cloud neural network based on octree structure, termed *SD-PCNet*, which abstracts a brand-new processing flow for point clouds by utilizing spatial similarity with density-adaptive subtrees and dynamic differential execution. The network consists of five stages: 1) density-adaptive subtree selection; 2) similarity-aware neighbor search; 3) common neighbor identification; 4) dynamic differential update; and 5) dynamic differential aggregation. On the hardware side, we propose a supporting point cloud accelerator, dubbed *SD-PCAcc*, to convert the algorithm-level redundancy reductions into performance enhancements. Our accelerator has four core functional components, including a similarity-aware coordinator (Sa-Coordinator), a similarity-aware neighbor search unit (Sa-NSU), a differentiable update unit (DUU), and a differential aggregation unit (DAU). On the deployment side, we propose multiple strategies

to deploy *SD-PCNet* on *SD-PCAcc* in a well-balanced pipeline with optimized hardware utilization and good scalability. Extensive evaluation results show that, compared with Crescent [13], the state-of-the-art point cloud accelerator, *SimDiff* gains an average of $3.2\times$ speedup and $3.1\times$ energy efficiency.

This article significantly expands on our conference paper of GDPCA [14], introducing both technical enhancements and practical implementation considerations. We now address the uneven distribution of points in point clouds by using an octree instead of the voxel-based partitioning from our previous work. The octree allows for balanced partitioning through a density-aware subtree selection strategy, which is more hardware-friendly. Additionally, we have developed a subtree-centric workflow that reduces memory access and turnaround delay, incorporating efficient data reuse and optimized parallelism. We also provide detailed quantitative analyses of our accelerator’s specifications, supporting the rationale behind its configuration. For deployment, we introduce strategies like network-to-accelerator mapping, core component reconstruction, and high-bandwidth memory (HBM) channel allocation to enhance feasibility and scalability. Finally, our framework’s effectiveness is validated across more datasets and application scenarios.

The remainder of this article is organized as follows. Section II introduced background and related work. Section III analyzed the opportunities and challenges. Section IV proposed our *SimDiff* framework, which is composed of a point cloud network *SD-PCNet* in Section IV-A, a supporting accelerator *SD-PCAcc* in Section IV-B, and multiple strategies for efficient network-to-accelerator deployment in Section IV-C. Performance evaluations were shown in Sections V and VI drew conclusion and look ahead to future work.

II. BACKGROUND AND RELATED WORK

A. Point Cloud Neural Networks

As depicted in Fig. 1, a point cloud image consists of an unordered set of points $X = x_1, x_2, \dots, x_n$, where each $x_i \in \mathbb{R}^F$ includes its spatial coordinate (x, y, z) and associated features. Neural networks like PointNet++ [7] and DGCNN [6] extract these features to make predictions, and their standard processing involves three stages per layer: Neighbor Search, Edge Update, and Neighbor Aggregation.

1) *Neighbor Search*: Once the sampled point cloud is obtained based on downsampling techniques, such as farthest point sampling or random sampling, as done in [15] and [7], the neighbor search operation calculates and identifies the spatially adjacent neighbors for each target point in the cloud, using typical methods, such as K -nearest neighbor search (KNN) and ball query [7], [16].

2) *Edge Update*: After all the neighbors of each target point are known, we assume a “virtual” edge connects the target point and each of its neighbors. The edge update operation will first calculate the feature of every edge (i.e., subtracting the features between two endpoints of each edge), then update all the edge features using linear aggregation in

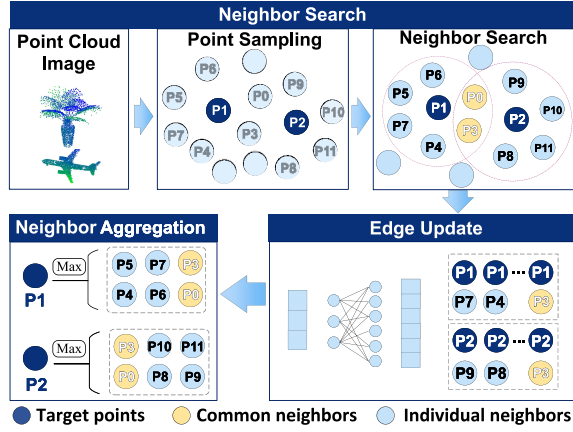


Fig. 1. Illustration of a layer of point cloud neural networks.

batch. Specifically, given the target points x_i and its neighbor $x_{j_k} \in N(x_i)$, $|N(x_i)| = K$, the edge update operation can be modeled as follows:

$$e_{ij_k} = \sigma(w \cdot (x_{j_k} - x_i)) \quad (1)$$

where e_{ij_k} is the updated edge feature, w is the learnable parameters, and σ is an activation function, e.g., ReLU.

3) *Neighbor Aggregation*: The neighbor aggregation operation derives new features for all target points by collecting the feature information from their neighbors. Max-pooling is commonly employed as the aggregation function, formulated as follows:

$$\hat{x}_i = \max_{1 \leq k \leq K} e_{ij_k}. \quad (2)$$

B. Point Cloud Accelerators

With recent progress in point cloud neural networks, well-suited accelerators [11], [12], [13], [17] flourished since 2020, aiming to boost these networks' processing. Mesorasi [11] is the first point cloud accelerator designed for efficient point cloud processing. PointAcc [12] addressed the sparsity challenge in point clouds. It caches sparse calculations and fuses dense computational layers on demand, offering compatibility with main-stream point cloud networks. Point-X [17] leverages spatial locality to boost communication efficiency in graph-based DNNs for point clouds, especially in DGCNN. Crescent [13] introduced a novel micro-architecture designed to manage memory irregularities, thereby achieving point cloud acceleration. However, these accelerators ignore the inherent similarities between spatially adjacent points in point clouds and the resulting significant redundancy. From a brand-new perspective, there is a huge potential for point cloud acceleration, i.e., to exploit the similarities to reduce such redundancy.

III. ANALYSIS AND CHALLENGES

A. Analysis of Spatial Similarity in Point Cloud Networks

According to extensive experiments and quantitative analysis, we reveal two main spatial similarities in point cloud networks: 1) feature similarity and 2) neighbor similarity.

Spatial Feature Similarity denotes that the features of spatially proximate points are highly similar. We set a target point c as the central point and its spatial feature is x_c . For every point j that is spatially around c , its spatial feature x_c can be modeled in a differential form as (3), based on the delta method proposed in [18]

$$x_j = x_c + (x_j - x_c) = x_c + \Delta_{c,j} \quad (3)$$

where $\Delta_{c,j}$ represents the element-wise differences between x_c and x_j . A small $\Delta_{c,j}$ indicates strong spatial correlations between two points.

Here offers an opportunity to reduce the required computational workload, as small deltas require fewer effectual bits to represent, compared to the raw features. We define Effectual Bits as follows: in the multiplication $x_j \times w$ of node x_j and weight w . If x_j is represented by m bits, its multiplication with w equates to summing p terms, with the m th term resulting from multiplying the m th bit of the multiplier x_j with the shifted m bit positions multiplicand w , formulated as (4). As only the logic "1" bits in x_j contribute to effective computation, the number of effectual bits corresponds to the count of the logic "1" in x_j

$$x_j \times w = \sum_{m=0}^{m=p} x_{j_m} \cdot (w \ll m). \quad (4)$$

We conducted experiments by comparing raw features with a multi-central point Δ representation. Point clouds were partitioned based on [19]. As shown in Fig. 2, the cumulative distribution of effectual bits across DGCNN layers demonstrates that, with preserved accuracy, the Δ representation significantly increases feature sparsity, especially in deeper layers, offering potential for reduced computation costs.

Spatial Neighbor Similarity denotes that spatially proximate points share a significant number of common neighbors. We define Neighbor Similarity NS as the proportion of the common neighbor set of all points in a cloud to all the neighbors, formulated as

$$NS = \frac{|\bigcap_{x_i \in \{x_1, \dots, x_s\}} N(x_i)| \cdot s}{\sum_{x_i \in \{x_1, \dots, x_s\}} |N(x_i)|}. \quad (5)$$

Here, $\bigcap_{x_i \in \{x_1, \dots, x_s\}} N(x_i)$ represents the common neighbor set of the cloud that contains s points.

Fig. 3 shows the average neighbor similarity of DGCNN on the KITTI dataset across various layers. The results validated our claim. Specifically, neighbor similarity exceeds 75.6% across all the layers, up to 90.1% in Layer 3. That is, a large percentage of neighbors are common neighbors for spatial adjacent points. This offers a valuable chance to enhance memory access efficiency by reusing the features of shared neighbors during the neighbor aggregation procedure.

B. Challenges of Exploiting Spatial Similarity for Point Cloud Acceleration

Although the similarities of spatial adjacent points exists objectively, several key obstacles hinder the use of spatial similarity to accelerate point clouds.

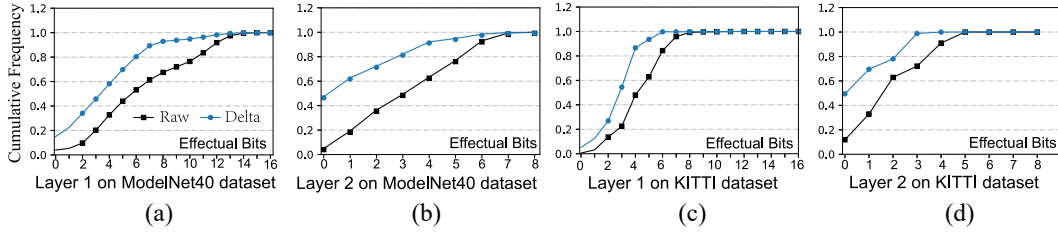


Fig. 2. Cumulative distribution of the number of effectual bits when using raw features and delta features based on the DGCNN model and KITTI and Modelnet40 datasets. (a) Layer 1 on ModelNet40 dataset. (b) Layer 2 on ModelNet40 dataset. (c) Layer 1 on KITTI dataset. (d) Layer 2 on KITTI dataset.

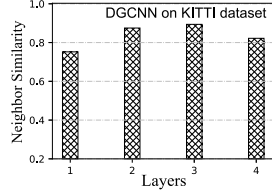


Fig. 3. Average neighbor similarity across various layers.

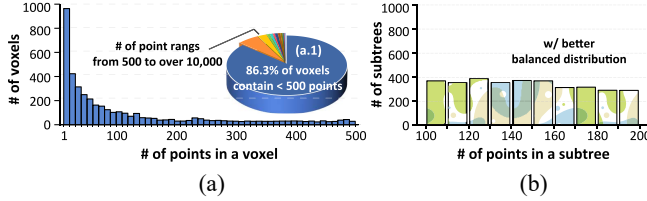


Fig. 4. Point density distribution based on (a) voxel-based partitioning and (b) subtree-based partitioning.

- 1) The edge update operation in the standard processing flow of point cloud networks breaks both the feature similarity and the neighbor similarity between adjacent points. First, the edge update operation involves subtraction between the source and the target points in feature space, inducing a shift in feature values and disrupting spatial feature similarity. Second, despite numerous common neighbors between adjacent points, the edge update operation is performed for each point pair separately. Consequently, spatial neighbor similarity cannot be leveraged at the neighbor aggregation stage. We design *SD-PCNet*, a novel similarity-aware differential point cloud neural network to leverage spatial similarity without compromising accuracy (see Section IV-A).
- 2) The unevenly distributed nature of points in point clouds makes similarity-aware computations hardware-unfriendly. Point clouds are naturally uneven, the density of the radar point cloud is much higher in the central area than in the edge area. The state-of-the-art approach [14] utilized voxels to partition the space and leverage the spatial similarity within the voxels. As voxels are intrinsically uniform in scale, dense voxels contain many more points than sparse voxels. As shown in Fig. 4(a), the number of points in a voxel varies from 1 to over 10 000. When processing in ASICs, this unevenness in input partitions will lead to disparity in computation

and on-chip resource requirements, making it hardware-unfriendly. Unlike voxel-based partitioning, partitioning point cloud space using octree (i.e., subtree) [19] offers greater flexibility because the subtrees are allowed to have flexible scales. To efficiently process point clouds with uneven point density, we propose a density-adaptive subtree selection strategy that can flexibly adjust the scale of subtrees according to the point density in space, allowing each subtree to contain roughly the same number of points [i.e., with balanced distribution, as shown in Fig. 4(b)], paving a good path to design a well-balanced accelerator.

- 3) Lack of appropriate algorithm and hardware co-design to effectively support spatial similarity exploitation. Most existing point cloud accelerators are designed based on the standard processing flow [11], [12], [13]. Nevertheless, leveraging spatial similarity entails a brand-new processing flow with sophisticated operations. Many problems are required to be solved. For example, how to efficiently identify common neighbors and how to reuse them? What are the design of hardware units to support these sophisticated operations? How to achieve well-pipelined scheduling. In this work, we propose a comprehensive framework that integrates advanced algorithms, hardware architectures, and deployment strategies to efficiently exploit spatial similarity.

IV. PROPOSED *SimDiff* FRAMEWORK

As illustrated in Fig. 5, *SimDiff* is a systematic software-hardware co-design framework, focusing on end-to-end leveraging spatial similarities to accelerate the execution of point cloud neural networks. It contains three enablers.

The first enabler is a similarity-aware differential point cloud neural network, dubbed *SD-PCNet*, which reduces computation and memory access redundancies by utilizing the spatial similarities in point clouds. The network abstracts the processing flow into five stages, including density-adaptive subtree selection based on octree partitioning, similarity-aware neighbor search, common neighbor identification, dynamic differential update, and dynamic differential aggregation.

The second enabler is *SD-PCAcc*, a pairing point cloud accelerator to support our network, converting redundancy reduction into actual performance enhancement. Correspondingly, our accelerator contains four core functional units, including 1) a Sa-Coordinator that is responsible for

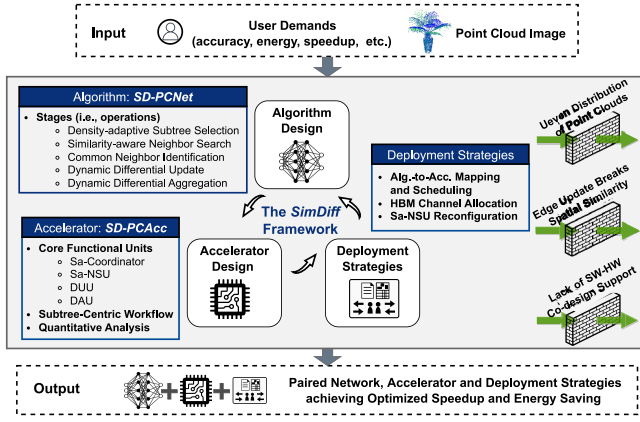


Fig. 5. Proposed SimDiff framework.

subtree selection and network-to-accelerator deployment; 2) a Sa-NSU that takes charge of neighbor search and common neighbor identification; 3) a differential update unit; and 4) a DAU that are, respectively, designed to update and gather features of points based on delta representation.

The third enabler is strategies to efficiently deploy *SD-PCNet* on *SD-PCAcc*, including a network-to-accelerator allocation and scheduling strategy, an HBM channel allocation strategy and a reconfiguration strategy for the Sa-NSU, facilitating the proposed methodologies into practical implementation.

A. SD-PCNet: Similarity-Aware Differential Point Cloud Neural Network

Overview and workflow of SD-PCNet: As shown in Fig. 6, differing from the standard processing flow of existing point cloud neural networks, our *SD-PCNet* proposes a brand-new execution abstraction based on dynamic differential representation. Specifically, given an input point cloud X , we first build an octree T using the method as in [20], which is the input of our network. Five stages together form a layer in our *SD-PCNet* model. At the stage of ① *density-adaptive subtree selection*, the octree (i.e., the entire point cloud) is partitioned into multiple spatially proximal subtrees. The processing order (i.e., priority) of these subtrees is determined as well. At the stage of ② *similarity-aware neighbor search*, within each spatial proximal subtree, we search for the neighbors for given target points. Then at the ③ *common neighbor identification* stage, we find the common neighbors of all the points in every subtree, forming a common neighbor set. As the points in the common neighbor set are shared neighbors of every point in the same subtree, their features only need to be calculated once and then can be reused without repeated calculations. In this way, the neighbor similarity is utilized to reduce computation redundancy at this stage. After that, at the stages of ④ *dynamic differential update* and ⑤ *dynamic differential aggregation*, we utilize the feature similarity of the points in a subtree to further reduce the computation and memory access costs based on delta representation and the methodology of delayed aggregation. After being executed layer by layer, our network

Algorithm 1 Density-Adaptive Subtree Selection Algorithm

```

1: initialization
   Initialize subtreeList
2: for leaf_node in leafNodeList do
   if leaf_node has not been traversed then
4:     current_node ← leaf_node;
     Append leaf_node to newSubtree
6:     Mark leaf_node as traversed;
   end if
   < depth-first
8:   while point number of newSubtree < M do
     Update current_node by backtracking on last current_node;
10:    Append current_node to newSubtree;
     Mark current_node has been traversed;
12:   end while
   < breadth-first
14:   for node in siblingNodes of current_node do
     if node has not been traversed and point number of newSubtree
       < M then
16:       Append node to newSubtree;
       Mark node as traversed;
     end if
18:   end for
   Append newSubtree to subtreeList;
20: end for

```

is completed and the final result is output. Here, we detail these stages.

① **Density-Adaptive Subtree Selection:** When employing an octree representation for point clouds, neighboring points are typically located within a subtree of lesser depth. To validate this hypothesis, we first define the proximate subtree as follows:

Definition 1 (Spatial Proximal Subtree): Given an input octree T and every leaf node L is set at the deepest depth of each local part of the octree. A spatial proximal subtree τ is defined as a subset of T that consists of neighboring points. It is typically formed by a complete subtree and selected portions of the sibling nodes of its root node. Formally

$$\tau_{(L,d,S)} = B_d(L) + S \quad (6)$$

where B_d represents the backtracking with the depth of d , which can obtain a complete tree, and the S denotes the subset of the sibling points of the complete tree's root node. As shown in Fig. 6, the subtree (a) is a complete tree, which can be obtained by backtracking on any point in the Depth3 by a backtracking depth of 2, and it can further form another spatial proximal subtree with any subset of nodes $\{ '3', '1' \}$ in Depth1. For validation, Fig. 7 collected the cumulative distribution of neighbors in various tree depths based on KITTI and ModelNet40 datasets, which reveals a strong concentration of point neighbors within 3-depth in ModelNet40 dataset and slightly weaker within 5-depth in KITTI dataset, and searching neighbors within this range will not compromise task accuracy.

Based on the above definition, we further design the hardware-efficient subtree search method to identify spatially proximal subtrees and determine their execution order. It ensures that the points contained in each subtree are within a precisely calculable range, adapting to point clouds with various densities. In addition, the points within all subtrees can cover the entire space.

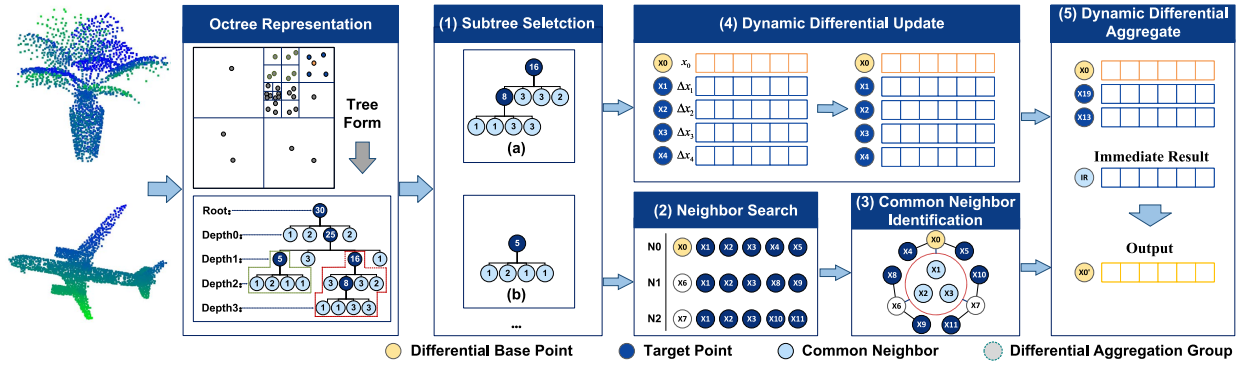


Fig. 6. Illustration of *SD-PCNet*. It contains five main stages: density-adaptive subtree selection, similarity-aware neighbor search, common neighbor identification, dynamic differential update, and dynamic differential aggregation.

As shown in Algorithm 1, we utilize a hybrid depth-breadth search to traverse all the leaf nodes while also optimizing off-chip memory access by reusing the overlapped leaf node of subtrees. The process begins from an untraversed leaf node and proceeds to backtrack to its parent nodes, until it encounters a node where the total number of points contained within all its leaf nodes exceeds a predefined threshold M . Following this, sibling nodes of the encountered node that have a point count less than M are traversed. The subtree of the encountered node and all the traversed sibling nodes are marked as traversed and constitute the spatial proximal subtree τ to be returned. The subtrees executed in sequence are spatially proximate, potentially involving overlapping leaf nodes. This sequence allows the hardware to optimize off-chip memory access by reusing the overlapped leaf nodes of the subtrees.

The number of points in each subtree has a defined upper and lower limit. The lower bound is M , while the upper bound is $M + N_l$, where N_l represents the maximum number of leaf nodes in the subtree, which is usually determined during the tree construction. This is because once the addition of a sibling leaf node causes the number of points in the subtree to exceed M , the subtree is yield.

② Similarity-Aware Neighbor Search: We restrict the search space as the spatial proximal subtree τ , which is selected through Algorithm 1. Given a subtree τ , the neighbors of a target point x_i are computed by the Euclidean distance

$$j_k = \arg \min_{x_j \in \tau} D(x_i, x_j) = \arg \min_{x_j \in \tau} \|x_i - x_j\|^2 \quad (7)$$

where $j_k \in \mathbb{R}^K$ denotes the k th neighbor index of point x_i . In this work, for every target point, we obtain its neighbor map using KNN.

③ Common Neighbor Identification: As analyzed in Section III-A, spatially proximate points share a significant number of common neighbors. We exploit this spatial neighbor similarity within each spatial proximal subtree. This stage aims to identify the common neighbors of all the target points in τ , i.e., the common neighbor set AG_τ to facilitate subsequent aggregate operations to reuse the intermediate computation results of common neighbors. The definition is as follows:

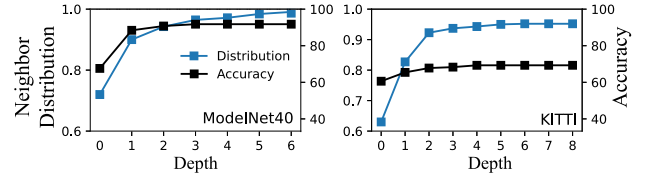


Fig. 7. Neighbor distribution in various tree depths based on KITTI and ModelNet40 datasets. The accuracy is measured with classification task on ModelNet40 with DGCNN and the segmentation task on KITTI with PointNet++.

Definition 2 (Common Neighbor Set): Given a spatial proximal subtree τ , its common neighbor set AG_τ can be defined as the intersection of neighbors of all points in τ , i.e., $AG_\tau = \{\bigcap_{x_i \in \tau} N(x_i)\}$.

④ Dynamic Differential Update: To further exploit spatial feature similarity, we design a dynamic differential update operation by utilizing delta representation. We first introduce two definitions:

Definition 3 (Differential Update Group): Given a spatial proximal subtree τ , the subset of points in τ that have not been updated is termed as a differential update group UG_τ .

Definition 4 (Differential Base Point): Given a differential update group UG_τ , we compute the center p_c of UG_τ based on the position, and the differential base point x_c can be defined as the point nearest to p_c in UG_τ .

For a given differential update group UG_τ , we first update the differential base points x_{c_i} using raw features. The subsequent points $x_\tau \in UG_\tau$ are computed using deltas, which possess fewer significant bits, enhancing computational efficiency. The process is defined in

$$\begin{aligned} \bar{x}_c &= w \cdot x_c \\ \bar{x}_\tau &= w \cdot x_\tau = w \cdot (x_c + \Delta_\tau) = w \cdot x_c + w \cdot \Delta_\tau. \end{aligned} \quad (8)$$

⑤ Dynamic Differential Aggregate: This stage produces new features by aggregating feature information from all the neighbors.

We propose a dynamic differential aggregation operation to exploit the spatial neighbor similarity. First, we are motivated by the methodology of delay aggregation as done in [11] and [17], which approximates the max aggregation

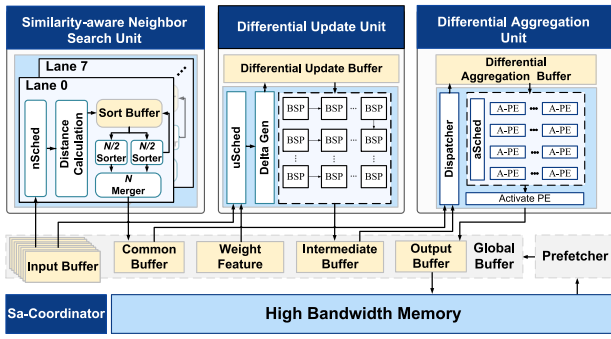


Fig. 8. Overview of the proposed *SD-PCAcc*.

operation in (1) and (2) as follows:

$$\hat{x}_i = \max_{1 \leq k \leq K} \sigma(w \cdot (x_{jk} - x_i)) \approx \sigma \left(\max_{1 \leq k \leq K} (w \cdot x_{jk}) - w \cdot x_i \right). \quad (9)$$

This approximation enables the removal of edge update operations, and thus decouples neighbor search operations and feature update operations, allowing them to be executed in parallel for greater efficiency.

As for common neighbor set reuse, we initially aggregate the common neighbors in AG_τ , buffering their aggregation results $\hat{x}_{com}$ for future use. Then, a differential aggregation is performed for each target point by utilizing the buffered aggregation results $\hat{x}_{com}$. Given a target x_i to be aggregated along with its neighborhood information $N(x_i)$, the dynamic differential aggregation operation can be modeled as follows:

$$\begin{aligned} \hat{x}_{com} &= \max_{j \in AG_\tau} (\tilde{x}_j) \\ \hat{x}_\tau &= \max \left(\hat{x}_{com}, \max_{m \in (N(x_i) - AG_\tau)} (\tilde{x}_m) \right) \end{aligned} \quad (10)$$

where $\hat{x}_{com}$ is the aggregation results of the common neighbor set AG_τ . New target features will be obtained, constituting the final outcome.

B. *SD-PCAcc*: Similarity-Aware Differential Point Cloud Accelerator

Overview: As illustrated in Fig. 8, *SD-PCAcc* is comprised of four core units to efficiently support our *SD-PCNet*, including a Sa-Coordinator for subtree selection and network-to-accelerator deployment control; a Sa-NSU for neighbor search and common neighbor identification; a DUU for feature update based on delta representation; and a DAU to perform aggregation with online reuse of the common neighbor set. Multiple buffers and an HBM are used for data caching and storage. On top of these hardware, an efficient subtree-centric workflow is further proposed toward superior hardware utilization and acceleration efficiency.

Subtree-Centric Workflow: Given input point cloud images and a well-trained *SD-PCNet* where the images are represented into octrees. For each layer of the *SD-PCNet*, Sa-Coordinator starts by selecting proper subtrees and prefetches the required points features from HBM into the Input Buffer. Sa-NSU loads the point features from the Input Buffer and executes both the neighbor search and common neighbor identification stages.

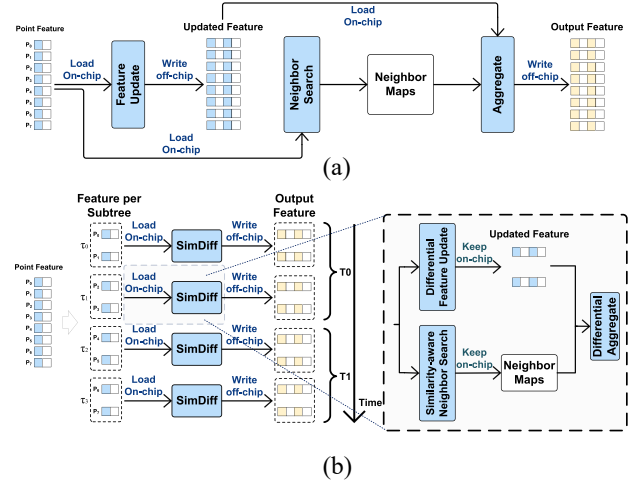


Fig. 9. (a) Traditional workflow to leverage spatial similarity. (b) Our subtrees-centric workflow. For all the points, our method reduces at least one off-chip memory read of the origin and updated features, and at least one off-chip memory write of the updated features.

The execution results for each subtree, including the raw feature of base points, the neighbor maps, and the identified common neighbor sets, are all cached in the Common Buffer. In parallel, DUU also loads the raw features and weights from the Input Buffer and Weight Buffer, respectively, and performs differentials calculation and feature update operations. The execution results (i.e., the updated features) are cached into the Intermediate Buffer. For each subtree, once Sa-NSU and DUU both finish execution, DAU starts to work. On the first execution of DAU, it will load the common neighbors (which have been identified in the Sa-NSU before) from the Common Buffer and calculate the aggregation result of all the common neighbors. The aggregation result are saved in the private register in DAU, waiting to be reused. After that, DAU collects the updated feature from the Intermediate Buffer, Combined with the aggregation results, and generates new point features, which are then written back to the Output Buffer. After layer by layer, our *SD-PCAcc* obtains the final features of the point clouds. Later, such learned point cloud representations can be transferred to various downstream tasks.

Here, we highlight two main advantages of our workflow. For better understanding, we illustrated the traditional workflow and our subtree-centric workflow in Fig. 9(a) and (b), respectively.

1) **Data Movement Reduction:** Traditional delay aggregation-based workflow utilize use voxel partitioning [14] or no partitioning [11] schemes. Due to the unevenness of the point cloud, the amounts of these bulk data even with voxel partition may be beyond the capacity of on-chip buffers. So for each stage, all its input features are loaded from off-chip memories and the intermediate results are to be written off-chip without cross-stage on-chip reuse, resulting in expensive data movement overhead. In contrast, in our workflow, we partition the entire point cloud space into multiple subtrees, and the proposed subtree selection algorithm

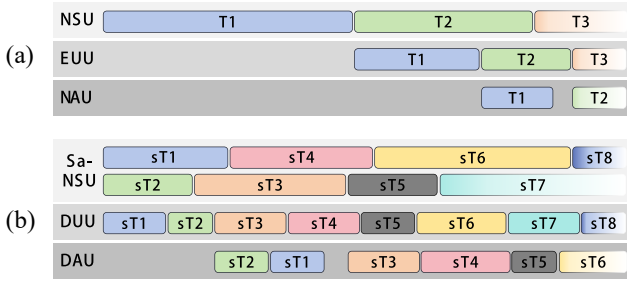


Fig. 10. Execution timeline of (a) workflow of the baseline accelerators, and (b) our proposed subtree-centric workflow. T is the workload in the traditional point cloud neural network, and sT is subtree partitioned workload in our workflow. We archive higher throughput with we-design parallel and balanced pipeline, that feature update (for EUU and DUU) and aggregation (for NAU and DAU) execute independently, thus overlapping the execution time of A and B.

ensures the point features will not exceed the buffer sizes. It is feasible to specify that all data for subtrees are stored in on-chip buffers before all the points in a subtree are completed with trivial and even no data movement costs.

- 2) *Timing Pattern Optimization Based on Well-Designed Parallel and Balanced Pipeline*: Baseline accelerators, such as PointAcc, adopt a stage-by-stage (i.e., sequential) workflow [shown in Fig. 9(a)]. In contrast, in our workflow, for each subtree, the Sa-NSU and the DUU work in parallel. Then, DAU polls a ready subtree that finishes the neighbor search and feature update processes. Fig. 10 gives an example to compare the timing patterns when adopting the two workflows. We assume there are two lanes in a Sa-NSU. It shows that, all of the core functional units (e.g., Sa-NSU, DUU, and DAU) achieve the maximal utilization and high throughput based on a well-balanced pipeline. Based on in-depth analysis, we implement eight computation lanes with subtree-level parallelism in a Sa-NSU to match its processing speed with the DUU. Detailed scheduling analysis refers to Section IV-C.

Core Functional Units: Here, we detail the micro-architectures, execution flows, and design highlights of the four core units in *SD-PCAcc*.

① *Sa-Coordinator*: The *Sa-Coordinator* functions as a co-processor. It takes charge of selecting spatial proximal subtrees as per Algorithm 1, and *SD-PCNet*-to-*SD-PCAcc* mapping and scheduling as per Algorithm 2, HBM channels allocation, and Sa-NSU reconfiguration configuration. Details of the latter three strategies are discussed in Section IV-C.

② *Sa-NSU*: A Sa-NSU contains multiple processing lanes, which execute in parallel to boost the search efficiency. Each lane processes an independent subtree, and occupies an exclusive input buffer and dedicated HBM channels for improved bandwidth utilization and to avoid access conflicts. The lanes and their input buffers can be reconfigured to process applications with larger subtrees. (The reconfiguration strategy refers to Section IV-C.) Within a lane, there are several modules. *nSched* is designed to retrieve neighbor search workloads from the Input Buffer. The Distance Calculator is

used to compute the distance, and the results are cached in the Sort Buffer. The Sort Buffer is divided into two parts to facilitate the online generation scheme: 1) the Neighbor Slot and 2) the Candidate Neighbor Slot, storing the Neighbor Array and Candidate Neighbor Array, respectively. Two $N/2$ Sorters and an N merger are utilized for sorting and conducting the top- k operation.

Execution Logic of Sa-NSU: For each subtree UG_τ , we select a target point x_i and generate its initial neighbor set AG_τ in Common Buffer. The workload is submitted via *nsched*, consisting of two stages: i) Access points in AG_τ , calculate distances, and store results in the Neighbor Slot; ii) Process remaining points in R_{x_i} in the Candidate Neighbor Slot. As shown in Fig. 11(c), sort and top- k operations are performed using two $N/2$ Sorters and an N Merger. A DMUX verifies if processing is complete, and once both slots are sorted, interpolation-merging is carried out, updating AG_τ . This process repeats for all points in UG_τ until all leaf nodes are processed.

Design Highlights in Sa-NSU: It is noteworthy that our Sa-NSU seamlessly combines the neighbor search operation and the common neighbor identification operation into a whole with ingenuity, largely decreasing hardware resource costs and calculational energy and time overheads. Commonly, a straightforward method for neighbor search and common neighbor identification is that we first traverse all the points in UG_τ to find each point's neighbors independently. After that, we use a merge search to determine the common neighbors (i.e., AG_τ). However, this approach significantly increases merge search operations and adds extra computational cycles due to the post-processing merge search. In the Sa-NSU, we developed an online generation scheme that iteratively updates and generates AG_τ on-the-fly without adding extra clock cycles or hardware resources. Fig. 11(a) demonstrates the execution process. Given a subtree UG_τ , we first conduct a similarity-aware neighbor search on the differential base point x_{c_i} and initialize AG_τ with the search results, namely, the neighbor index of x_{c_i} . Next, we choose a point $x_i \in \{UG_\tau - x_{c_i}\}$ for further processing. Here, x_i 's search region R_{x_i} is divided into two sets: the current AG_τ and $\{R_{x_i} - AG_\tau\}$. We then calculate and sort the distances of all points in AG_τ to x_i in ascending order to form the neighbor array. And further, a similar process is applied to the set $\{R_{x_i} - AG_\tau\}$, using the top- k operation to select k points for the candidate neighbor array. Next, an interpolation-merging operation is used to combine the two arrays, which dynamically updates AG_τ with only $O(1)$ time complexity. Last, we traverse the remaining points in UG_τ , repeating such steps until all points in UG_τ are processed, resulting in the final AG_τ .

③ *Differential Update Unit (DUU)*: The Differential Update Unit (DUU) is designed to handle dynamic differential updates. It consists of a *uSched*, a Delta Generator, a Differential Update Buffer, and a Differential Update Processing Unit (DU-PU). The *uSched* retrieves updated workloads, while the Delta Generator computes deltas through element-wise subtraction. These deltas are processed by the DU-PU, which is composed of Offset Generators and Basic Processing Units (BSPs). Each BSP column processes point

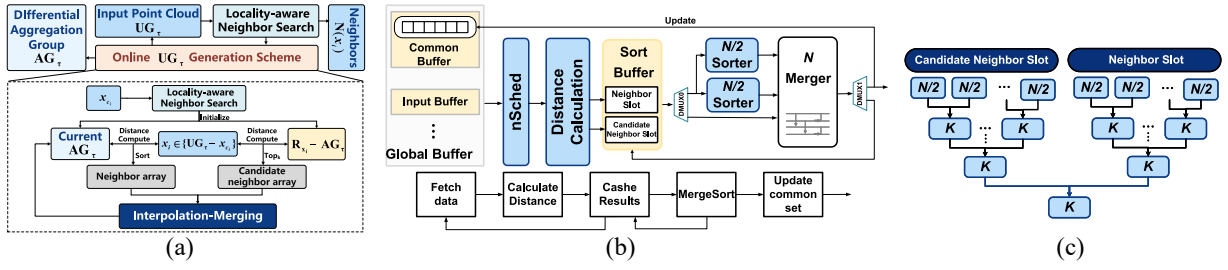


Fig. 11. (a) Illustration of seamless online generation of common neighbor set AG_τ . (b) Design of a lane in Sa-NSU. (c) Neighbor search workflow in Sa-NSU.

features by transforming them into binary streams and performing efficient updates using shifters and adder trees. A differential reconstruction element (DRP) restores features after delta processing, and a MUX system allows for switching between deltas and raw values. The execution starts with $uSched$ fetching the first set of points, and the Delta Generator transforms these into deltas for processing. BSPs in column 0 handle raw features, while the remaining columns process deltas. After processing, the results are stored and reconstructed into standard feature values. The State Register and MUX help manage the processing flow for efficient updates. For more details, please refer to our conference version.

Design Highlights in DUU: In addition to the design idea of using delta representation replacing raw feature for update operations in *SD-PCNet*, which is at the algorithm level, we emphasize one design ingenuity in DUU, i.e., we designed a DU-PU based on bit-serial processing. As analyzed in Section IV-A, using delta representation for point cloud processing occupies fewer effectual bits than using raw features for point cloud processing. As the delay and the energy consumption of the bit-serial processing element are both positively related to the number of effectual bits, it is an efficient choice to build DU-PU based on bit-serial processing, which can transfer the effectual bit reduction into the improvement of time and energy efficiencies.

④ **DAU:** This unit is designed to achieve the proposed differential aggregation operations, in which the common neighbor set is loaded only once to obtain their aggregation result. This result is reused in the next executions to eliminate redundant aggregations. Fig. 12 shows the architecture of DAU, comprising a Dispatcher, a Differential Aggregator Buffer, and a differential aggregation processing unit (DA-PU). The Dispatcher allocates the aggregation workload from the Common Buffer and Intermediate Buffer. The Differential Aggregator Buffer stores the workload for the aggregate task, consisting of multiple task slots. Each slot holds the workload for a point awaiting aggregation. The task slots have buffers to store the neighbor features. Each neighbor feature is linked to a Task Type Register indicating whether the neighbor belongs to the common neighbor set AG_τ (0: false, 1: true), and a State Register, which identifies if this point is the differential base point x_{ci} (0: false, 1: true). The DA-PU retrieves and processes aggregation tasks from the Differential Aggregator Buffer. It is equipped with an $aSched$, an aggregation processing element (A-PE) group, a Base Register, and two MUXs to manage the control logic. The $aSched$ orchestrates the pipeline execution

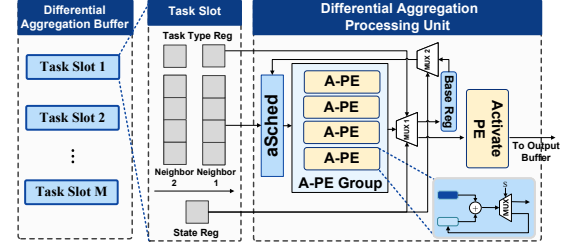


Fig. 12. Design of DAU.

and distributes tasks to the A-PE group, which handles the Max aggregate operation. MUX1 caches the intermediate results of AG_τ , and MUX2 is employed for reusing these results.

Execution Logic of DAU: It comprises two distinct phases.

- 1) **Aggregation Workload Preparation:** The Dispatcher feeds the aggregated workload into the Differential Aggregator Buffer, based on the outcomes of the first two stages. The workload for x_{ci} is allocated to Task Slot 0 (State Reg = 1). Its neighbors are categorized into two groups: AG_τ (Task Type Reg = 1) and individual neighbors (Task Type Reg = 0). In contrast, the remaining task slots exclusively store the corresponding individual neighbors by truncating AG_τ .
- 2) **Aggregation:** $aSched$ polls the Task Type Reg and State Reg, subsequently directing them to MUX1 and MUX2. For x_{ci} (state reg is 1), if its neighbor belongs to AG_τ (Task Type Reg = 1), the intermediate result is cached after the A-PE group completes the workload, no saving is required. For the other points in UG_τ (Task Type Reg = 0), the intermediate results are directly reused. All individual neighbors are processed sequentially until processing is completed (Task Type Reg = 0). Finally, A-PEs execute the activation operation to generate the output feature.

Algorithm-Oriented Hardware Specification Analysis: We have identified that numerous hardware parameters significantly influence the performance of algorithms in practical applications. For example, the size of the buffer can limit the size of the subtree, thereby affecting the accuracy of the algorithm. Similarly, the number of lanes in Sa-NSU and the specifications of computing arrays in three computation units can impact pipeline execution efficiency depending on the number of points and feature channel counts in the

algorithm. This section will analyze the selection of hardware specifications in conjunction with actual algorithms.

Cycle Estimation: For a computation lane in Sa-NSU, it includes a fully pipelineable distance computation, sort, and merge stages. We employ the point-parallel high-throughput sorting method proposed in [21] to implement two $N/2$ sorters and one N merger in one lane. The average cycle time for a subtree can be estimated as

$$t_{\text{NSU}} = \left(\frac{6N}{p} + o(N/p) \right) \times \frac{2p^2}{N} \quad (11)$$

where N and p denote the N -key sequence and parallelism level in sorter, and P is the average number of points in τ . Note that it has a complexity of squaring the number of points.

For the DAU, where the number of A-PE units is represented by N_{ape} , the estimated cycle count is given by

$$t_{\text{DAU}} = \frac{PKd_{\text{out}}}{N_{\text{ape}}} \times (1 - r) \quad (12)$$

where r is the average neighbor similarity, and K is the number of neighbors.

Hardware Specification Analysis: We observe that for the vast majority of applications, such as DGCNN and Pointnet++, maintaining the number of points P in the subtree to just over a hundred is sufficient to prevent a decline in accuracy. Typically, the dimensions for d_{in} and d_{out} are 128, and K is 16. Considering these algorithmic parameters, we initially selected a generally applicable 64×64 in the DUU. Subsequent calculations indicated that with N_{ape} set to 128, the throughput of the DUU and DAU closely aligns. However, for the NSU, due to hardware design constraints, increasing N beyond P fails to yield additional speed improvements. Even if we let parallel factor $p = N$ to fully leverage the intralane point-level parallelism, the NSU's computation lane is still approximately $10\times$ slower than those in the other units. Consequently, we configured the number of lanes at 8 to ensure matched throughput across all three units. With such hardware specifications and characteristics of 32-bit floating numbers, the total bandwidth requirement for HBM is approximately 120 GB/s. Networks that require higher precision or have more extensive weights will demand even greater bandwidth. Also, in this configuration, equipping 2560 kB of on-chip SRAM supports the proposed subtree-centric workflow by caching all input features and intermediate results for eight subtrees.

C. Strategies for Deploying SD-PCNet on SD-PCAcc

Algorithm-to-Hardware Mapping and Scheduling Strategy: This strategy aims to assign/schedule different computation stages of *SD-PCNet* to the core functional hardware units of *SD-PCAcc* with a well-balanced pipeline. The scheduling algorithm is shown in Algorithm 2. The scheduling algorithm is executed in the Sa-Coordinator, which first employs Algorithm 1 to orderly place subtrees into the subtreeList. Each subtree in the list contains three tasks: neighbor search (i.e., NSU task), differential update (i.e., DUU task), and differential aggregation (DAU task), all marked as incomplete.

Algorithm 2 Scheduling SD-PCNet on SD-PCAcc

Require: Point cloud image T , *SD-PCNet*, *SD-PCAcc*
Ensure: Mapping and Scheduling Solution

```

1: subtreeList  $\leftarrow$  selectSubtree( $T$ )
2: initializeTasksStatus(subtreeList,
   ["NSUTask", "DUUTask", "DAUTask"], "Not Completed")
3: while notAllTasksCompleted(subtreeList) do
4:    $\triangleleft$  Schedule Similarity-aware Neighbor Tasks
5:   if anyIdleLane( $Sa - NSU$ ) then
6:     idleLanes  $\leftarrow$  findIdleLanesForNSU()
7:     NSUMappings  $\leftarrow$  allocateToIdleLanes(idleLanes,
       subtreeList)
8:     for each (subtree, lane) in NSUMappings do
9:       lane.inputBuffer  $\leftarrow$  prefetch(subtree.pointsFeature)
10:      dispatchNSUTask(subtree, lane) asynchronously
11:    end for
12:  end if
13:   $\triangleleft$  Schedule Differential Update Tasks
14:  if DUU.isIdle() then
15:    Subtree  $\leftarrow$  chooseSubtreeByNSUProgress(subtreeList)
16:    dispatchDUUTask(Subtree, DUU) asynchronously
17:  end if
18:   $\triangleleft$  Schedule Differential Aggregation Tasks
19:  if DAU.isIdle() then
20:    Subtree  $\leftarrow$  chooseSubtreeReadyForDAU(subtreeList)
21:    dispatchDAUTask(Subtree, DAU) asynchronously
22:  end if
23:  updateTaskStatus(subtreeList,  $Sa - NSU$ , DUU, DAU)
24: end while

```

Initially, the Sa-Coordinator checks for available lanes in the Sa-NSU. If an idle lane is found, the coordinator pairs it with the first subtree in the subtreeList whose NSU task is pending. Subsequently, it prefetches the features of the points in the subtree to the input buffer of the paired lane. The NSU task of the subtree is then asynchronously dispatched to the corresponding lane, allowing the Sa-Coordinator to proceed without waiting for task completion.

The scheduling of the DUU task is similar to that of NSU but diverges in its prioritization strategy. Unlike the NSU task, which employs a first-come-first-served (FCFS) policy to schedule subtrees from the subtreeList, the DUU task uses a priority-based scheduling policy. Subtrees that have completed their NSU tasks are given higher priority. If no subtree has completed its NSU task, the longest one in the NSU phase is assigned the highest priority. As there is only one computing unit for DUU, while NSU has eight lanes. Such a priority assignment scheme helps complete the NSU and DUU tasks of the same subtree as simultaneously as possible, thus enabling an earlier start to the DAU task and reducing the overall latency for subtree processing.

The scheduling of the DAU task involves the Sa-Coordinator selecting a subtree that has completed both its NSU and DUU tasks. Sa-Coordinator periodically checks and updates tasks and hardware unit status, and begins scheduling with designated event triggers.

Reconfiguration Strategy for Sa-NSU: Different applications have application-specific demands on their performances and overheads. To improve the scalability and adaptability of *SimDiff*, we propose a strategy for Sa-NSU reconfiguration. Specifically, according to the user demands on task accuracy, we can obtain the value of the hyperparameter M in Algorithm 1, with which we achieve the maximized speedup

TABLE I
BENCHMARK MODELS AND DATASETS

Dataset	Point Cloud Network	Application Domains	Scene
ModelNet40 [23]	PointNet++(c) [7]	Classification	Object
	DGCNN(c) [6]		
	LDGCNN [9]		
	DensePoint [8]		
semanticKITTI [24] (sKITTI)	PointNet++(s)	Segmentation	Outdoor
	DGCNN(s)		
	F-PointNet [10]		
S3DIS [25]	PointNet++(s3)	Segmentation	Indoor
KITTI [26]	F-PointNet(d)	Detetion	Outdoor

with the accuracy requirement satisfied. Once M is determined, we need to reconfigure the Sa-NSU to accommodate the subtrees with sizes ranging from M to $M + N_i$. If M is large (i.e., at least M points in the subtrees), the capacity of input buffers will become a bottleneck. Because our subtree-centric workflow dictates that all input partitions are stored in on-chip buffers (i.e., the Input Buffers) before a subtree is completed. Therefore, to adapt to large inputs, our Sa-NSU occupies the ability to merge 2/4/8 input buffers and lanes as needed. The total capacities of merged buffer and lanes are then used to perform calculations.

HBM Channel Allocation Strategy: Based on the above analysis, due to the computational complexity of squaring the number of points in neighbor search and the upper limit on the parallelism level of sorting units, the memory bandwidth of a single compute lane generally does not exceed 10 GB/s, which cannot efficiently utilize the high bandwidth of HBM to process a generally small subtree (with over 100 points being sufficient to ensure no reduction in accuracy).

To effectively utilize the parallelism afforded by HBM2, and considering that different computation lanes access distinct points, we independently allocate HBM channels to individual lanes to prevent access conflicts during parallel computation, similar to [22]. Among the 16 pseudo channels of HBM2, typically 10 channels are activated where each lane corresponds to one channel (8 lanes in total), one channel is assigned to weights and one is dedicated to outputs. If the channel is bottlenecked, the remaining six channels can also be activated and dynamically assigned to computation lanes in scenarios demanding low latency to maintain a well-balanced pipeline.

V. PERFORMANCE EVALUATION

Benchmarks, Datasets and Baselines: We evaluated *SimDiff* based on five point cloud network benchmarks and two well-known datasets, as detailed in Table I. For acceleration performance comparison, we compared *SimDiff* against five hardware baselines, including a CPU baseline featuring Intel Xeon E5-2680 v3 CPUs with 500-GB DRAM, a GPU baseline with NVIDIA Tesla A100, and three leading point cloud accelerators, namely, Crescent [13], PointAcc [12], and Mesorasi [11]. To ensure fair comparisons, we simulate these accelerator baselines based on the same TSMC 40-nm technology and assign our *SimDiff* and the baselines with the same computation resources and HBM storage in all experiments, as listed in Table II.

TABLE II
CONFIGURATION OF ACCELERATOR BASELINES

Accelerator	Mesorasi [11]	PointAcc [12]	Crescent [13]	Ours
Cores	64×64	64×64	64×64	64×64
Systolic Unit Array	Systolic MAC	Systolic MAC	Systolic MAC	BSP Array & 128 A-PEs
SRAM(KB)	2048	776	2048	2560
Area (mm^2)	14.2	15.7	13.6	16.9
Frequency	1GHz	1GHz	1GHz	1GHz
Memory	HBM 2	HBM 2	HBM 2	HBM 2
Bandwidth	256 GB/s	256 GB/s	256 GB/s	256 GB/s
Technology	40 nm	40 nm	40 nm	40 nm

Methodology: We measure the performance and energy efficiency of *SimDiff* using several tools. We build a validated cycle-accurate simulator for cycle count statistics. This simulator models the microarchitectural behavior of each module, integrating with Ramulator [27] for HBM timing estimation and command trace generation. For each module's power and area metrics, we implement them in Verilog and employ Synopsys Design Compiler (DC) with TSMC's 40-nm standard VT library for synthesis, estimating power consumption with Synopsys PrimeTime PX. On-chip memory's area and power are measured using CACTI [28]. We use quantization-aware training in PyTorch across all baseline models, reducing precision to 16 bits, which incurs minimal accuracy loss compared to 32 bits.

Accuracy: Our study focuses on two accuracy-affecting factors: 1) the depth factor in octree-based neighbor search and 2) the delayed aggregation operation. Fig. 13(a) illustrates the impact of delayed aggregation on network models. It shows that the accuracy remains intact despite the approximation process inherent in delayed aggregation.

Fig. 13(b) further analyzes the accuracy-latency tradeoff under various subtree depth factors, i.e., the depth used in the backtracking of octree-based neighbor search. Small values of depth indicate fewer points in the subtree. We observe that there are turning points in the datasets' depth, with 4 for KITTI and 3 for ModelNet40. Before these turning points, it is possible to tradeoff accuracy for higher speedup. However, after the turning points, deeper subtrees do not continue to enhance accuracy, which remains comparable to the accuracy obtained from the original KNN search across the entire space. This phenomenon occurs because, before the turning points, smaller subtrees involve less computational and memory access in the Neighbor search phase, albeit with a greater deviation in neighbor search. After the turning points, the neighbors are already sufficient to capture crucial local information. This also proves that *SimDiff* can achieve performance improvement without comprising accuracy.

Speedup and Memory Access Cost: For speedup comparison, we use the total number of execution cycles as the performance metric. As shown in Fig. 14, *SimDiff* achieved acceleration of 5.5 $\times$, 2.3 $\times$, 3.2 $\times$, 199.8 $\times$, and 5.3 $\times$ on average, compared to Mesorasi, PointAcc, Crescent, CPU, and GPU, respectively. Furthermore, *SimDiff* demonstrates an average memory reduction of 85%, 71%, and 65% compared with Mesorasi, PointAcc, and Crescent, respectively.

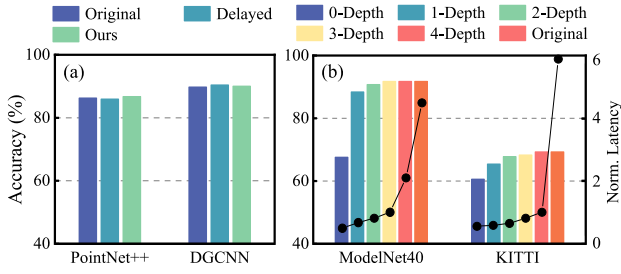


Fig. 13. Effect of accuracy determinants. (a) Delayed aggregation’s effect. (b) Accuracy–latency tradeoff with different depth factors.

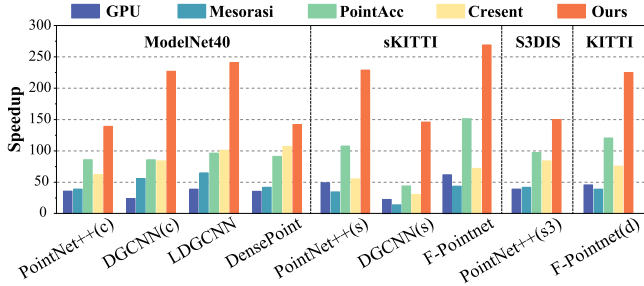


Fig. 14. Speedup comparison. All the results are normalized into that of the GPU baseline.

These performance enhancements are attributed to four key contributors. First, the Sa-Coordinator utilizes octrees to illustrate the spatial similarities in irregular point clouds, optimizing data reuse via subtree-level parallelism. Second, the Sa-NSU utilizes spatial similarity to minimize the number of neighbors loaded for searching and reduce the relative calculation, while the baseline accelerators must repeatedly load entire point features for mapping operation, incurring considerable memory access overhead. Third, our DUU capitalizes on spatial feature similarity by processing only the deltas (differences between features) rather than the raw features. This strategy utilizes the inherent feature similarity among points in nearby regions, leading to lower computational requirements. Last, our DAU reuses the aggregation results from common neighbor sets to reduce computation, and substitutes multiple accesses and aggregation for individual point features with singular access and aggregation to the group’s aggregated feature to reduce memory access.

Energy Consumption: Fig. 15 shows a normalized energy efficiency comparison between our method and other baseline accelerators. Overall, *SimDiff* achieves energy savings of $3.1\times$, $1.8\times$, and $5.0\times$ compared to Crescent, PointAcc, and Mesorasi, respectively. Fig. 16 illustrates the energy breakdown of *SimDiff* using two benchmarks: PointNet++ and DGCNN. More than 60% of the total energy is consumed by the computational engines. Additionally, energy overheads, like generating differential aggregation groups on the fly and offsets for deltas, are minimal compared to the energy savings.

Ablation Studies: We quantify and discuss the contribution of the *SimDiff*’s component techniques and units to provide more insights. Table III shows different variants of *SimDiff*

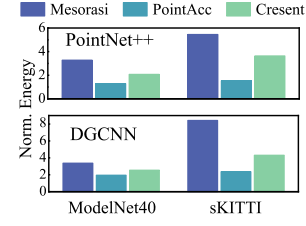


Fig. 15. Energy savings of *SimDiff* over baselines.

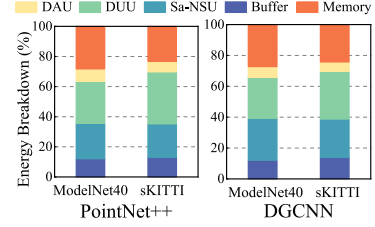


Fig. 16. Energy breakdown of *SimDiff* on benchmark models.

TABLE III
ABLATION STUDY SETUP FOR THE *SimDiff* FRAMEWORK

Notation	Similarity-aware Neighbor Search	Differential Update	Differential Aggregation	Subtree-centric Workflow
SimDiff-NS	×	✓	✓	×
SimDiff-DU	✓	×	✓	✓
SimDiff-DA	✓	✓	×	✓
SimDiff-SW	✓	✓	✓	×
SimDiff	✓	✓	✓	✓

and Fig. 17 illustrates the speedup and memory access reduction over these variants.

- 1) *Effects of Similarity-Aware Neighbor Search:* We compare *SimDiff* with its variant *SimDiff-NS* which searches neighbors throughout the entire point cloud space instead of within spatial proximal subtrees. Note that the subtree-centric workflow is not applicable in this variant. As shown in Fig. 17(a), the results show that *SimDiff* significantly reduces computation time and memory access. The performance enhancement is even more evident on the KITTI dataset, a real-world dataset with dense point clouds and better neighbor similarity than ModelNet40, underscoring its enhanced efficiency in accelerating the processing of large-scale point clouds in real-world applications.
- 2) *Effects of Dynamic Differential Update:* The performance of *SimDiff* is compared with its variant, *SimDiff-DU*, which uses a traditional systolic array in the update stage instead of the DUU. For a fair comparison, the systolic array in *SimDiff-DU* is configured to match the hardware specifications of the DUU. As shown in Fig. 17(b), our experiments reveal that the architecture employing DUU consistently outperforms the traditional systolic array variant in execution time. This improvement is due to the use of delta representation in the update operation, effectively reducing the number of bits processed compared to raw values. For example, in our 16-bit quantization setup, the average number of effectual bits for a 4-layer DGCNN is [13, 7, 7, 7] for ModelNet40, and [8, 5, 5, 5]

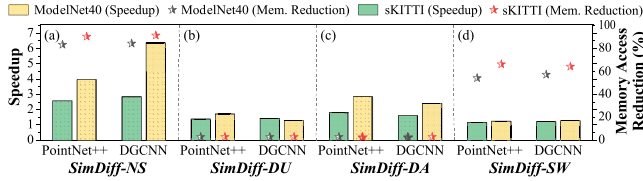


Fig. 17. (a) Effects of similarity-aware neighbor search. (b) Effects of dynamic differential update. (c) Effects of dynamic differential aggregation. (d) Effects of subtree-centric workflow.

for KITTI. Additionally, *SimDiff* shows greater speedup for KITTI, attributed to its strong local geometric correlations, resulting in higher feature similarity among neighbors. This leads to fewer effectual bits and reduced computation in delta processing for these points.

3) *Effects of Dynamic Differential Aggregation*: To test the effectiveness of our dynamic differential neighbor aggregation method, we created a *SimDiff-DA* variant for comparison, which does not reuse the aggregation results of the common neighbors. Fig. 17(c) reveals that our method demonstrates a notable speedup compared to *SimDiff-DA*. The advantage of our method lies in leveraging neighbor similarity and reusing the results within common neighbor sets, thereby reducing the computation required for aggregation.

4) *Effects of Subtree-Centric Workflow*: We compare *SimDiff* with the variant *SimDiff-SW* which adopts a delayed aggregation workflow in GDPCA [14]. As depicted in Fig. 17(d), *SimDiff* achieves an average speedup of 23% and memory reduction of 60%. This is because the dataflow of GDPCA contains multiple separate phases and suffers from reloading point features from off-chip to on-chip memory for execution in every phase, whereas *SimDiff* efficiently loads original features of all points in a subtree to the on-chip buffer at once, ensuring subsequent operations proceed without moving intermediate results between off-chip and on-chip memory, thus minimizing memory access. Additionally, unlike GDPCA's voxel representation, we employ an octree structure to depict spatial similarity, which facilitates balanced, conflict-free parallel access of multiple subtrees to HBM channels, enhancing memory bandwidth utilization and data reuse.

VI. CONCLUSION

This study demonstrates that nearby points show significant similarities and correlations within point clouds. Conventional implementations of point cloud neural networks often result in excessive computations and communications. To this end, we proposed an algorithm–hardware codesigned framework for point cloud acceleration. It contains a similarity-aware point cloud neural network and a supporting accelerator, capable of minimizing computation and memory access redundancies by leveraging spatial similarity and differential execution techniques. In our future work, in addition to technical improvements to the proposed *SimDiff*, we try to explore its integration within the larger computing landscape, including

the interoperability with cloud computing platforms, integrating with existing software framework, the adaptability to emerging point cloud formats, and the implications for privacy and security areas.

REFERENCES

- [1] Y. Cui et al., “Deep learning for image and point cloud fusion in autonomous driving: A review,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 722–739, Feb. 2022.
- [2] F. Pomerleau, F. Colas, and R. Siegwart, “A review of point cloud registration algorithms for mobile robotics,” *Found. Trends[®] Robot.*, vol. 4, no. 1, pp. 1–104, 2015.
- [3] Y. Wang, S. Zhang, B. Wan, W. He, and X. Bai, “Point cloud and visual feature-based tracking method for an augmented reality-aided mechanical assembly system,” *Int. J. Adv. Manuf. Technol.*, vol. 99, no. 9, pp. 2341–2352, Sep. 2018.
- [4] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12697–12705.
- [5] Y. Zhou and O. Tuzel, “VoxelNet: End-to-end learning for point cloud based 3D object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4490–4499.
- [6] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph CNN for learning on point clouds,” *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, 2019.
- [7] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5105–5114.
- [8] Y. Liu, B. Fan, G. Meng, J. Lu, S. Xiang, and C. Pan, “DensePoint: Learning densely contextual representation for efficient point cloud processing,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 5239–5248.
- [9] K. Zhang, M. Hao, J. Wang, C. W. de Silva, and C. Fu, “Linked dynamic graph CNN: Learning on point cloud via linking hierarchical features,” 2019, *arXiv:1904.10014*.
- [10] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, “Frustum PointNets for 3D object detection from RGB-D data,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 918–927.
- [11] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, “Mesorasi: Architecture support for point cloud analytics via delayed-aggregation,” in *Proc. IEEE/ACM Int. Symp. Microarchit.*, 2020, pp. 1037–1050.
- [12] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, “PointAcc: Efficient point cloud accelerator,” in *Proc. IEEE/ACM Int. Symp. Microarchit.*, 2021, pp. 449–461.
- [13] Y. Feng, G. Hammonds, Y. Gan, and Y. Zhu, “Crescent: Taming memory irregularities for accelerating deep point cloud analytics,” in *Proc. IEEE/ACM Int. Symp. Comput. Archit.*, 2022, pp. 962–977.
- [14] C. Chen, X. Zou, H. Shao, Y. Li, and K. Li, “Point cloud acceleration by exploiting geometric similarity,” in *Proc. IEEE/ACM Int. Symp. Microarchit.*, 2023, pp. 1135–1147.
- [15] Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen, “PointCNN: Convolution on X-transformed points,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–11.
- [16] C. Park, Y. Jeong, M. Cho, and J. Park, “Fast point transformer,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16949–16958.
- [17] J.-F. Zhang and Z. Zhang, “Point-X: A spatial-locality-aware architecture for energy-efficient graph-based point-cloud deep learning,” in *Proc. IEEE/ACM Int. Symp. Microarchit.*, 2021, pp. 1078–1090.
- [18] M. Mahmoud, K. Siu, and A. Moshovos, “Diffy: A Déjà vu-free differential deep neural network accelerator,” in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchit.*, 2018, pp. 134–147.
- [19] J. Elseberg, D. Borrmann, and A. Nüchter, “One billion points in the cloud—An octree for efficient processing of 3D laser scans,” *J. Photogramm. Remote Sens.*, vol. 76, pp. 76–88, Feb. 2013.
- [20] F. Chen, R. Ying, J. Xue, F. Wen, and P. Liu, “ParallelNN: A parallel octree-based nearest neighbor search accelerator for 3D point clouds,” in *Proc. IEEE/ACM Int. Symp. High-Perform. Comput. Archit.*, 2023, pp. 403–414.
- [21] R. Chen, S. Siriya, and V. Prasanna, “Energy and memory efficient mapping of bitonic sorting on FPGA,” in *Proc. ACM/SIGDA Int. Symp. Field-Programm. Gate Arrays*, 2015, pp. 240–249.

- [22] S. K. Venkataramanaiah et al., “FPGA-based low-batch training accelerator for modern CNNs featuring high bandwidth memory,” in *Proc. IEEE Int. Conf. Comput.-Aided Design*, 2020, pp. 1–8.
- [23] Z. Wu et al., “3D shapeNets: A deep representation for volumetric shapes,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1912–1920.
- [24] J. Behley et al., “SemanticKITTI: A dataset for semantic scene understanding of LiDAR sequences,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9297–9307.
- [25] I. Armeni et al., “3D semantic parsing of large-scale indoor spaces,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1534–1543.
- [26] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3354–3361.
- [27] Y. Kim, W. Yang, and O. Mutlu, “Ramulator: A fast and extensible DRAM simulator,” *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, pp. 45–49, Jan.–Jun. 2016.
- [28] S. Thoziyoor, N. Muralimanohar, and N. P. Jouppi, “CACTI 5.0,” Adv. Archit. Lab., Hewlett-Packard Develop. Co., Palo Alto, CA, USA, Rep. HPL-2007-167, 2007.



Yangfan Li received the Ph.D. degree in computer science from Hunan University, Changsha, China, in 2022.

He is currently a Lecturer with the School of Computer Science and Engineering, Central South University, Changsha. His research interests include computer architecture, efficient machine learning, and deep learning.



Mengquan Li received the Ph.D. degree in computer science from Chongqing University, Chongqing, China, in 2020.

She is an Associate Professor with the College of Integrated Circuits, Hunan University, Changsha, China. Her research interests include multiprocessor SoC, optical NoC, and photonic AI accelerator design automation.



Cen Chen (Senior Member, IEEE) received the Ph.D. degree in computer science from Hunan University, Changsha, China, in 2019.

He is currently a Professor with the School of Future Technology, South China University of Technology, Guangzhou, China. His research interests include parallel and distributed computing, machine learning, and deep learning.



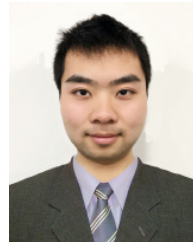
Xiaofeng Zou received the Ph.D. degree in computer science and technology from Hunan University, Changsha, China, in 2023.

His research interests include machine learning and deep learning.



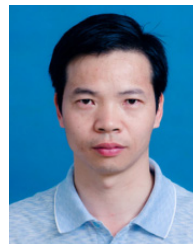
Hongen Shao is currently pursuing the Ph.D. degree with the College of Future Technology, Guangzhou International Campus, South China University of Technology, Guangzhou, China.

His main research fields include artificial intelligence algorithms and architecture design.



Fengxiao Tang (Member, IEEE) received the Ph.D. degree from the Graduate School of Information Science, Tohoku University, Sendai, Japan, in 2019.

He is a Full Professor with the School of Computer Science and Engineering, Central South University, Changsha, China. His research interests are IoT security, game theory optimization, network traffic control, and machine learning algorithms.



Kenli Li (Senior Member, IEEE) received the Ph.D. degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2003.

He is a Vice-President of Hunan University, Changsha, China. His research interests include parallel and distributed processing, high-performance computing, and big data management.